SHOPBOP SHOWDOWN

Backend Architecture Document

CS 620 Capstone | Team: ShopBop Showdown | Spring 2026

1. Project Overview

ShopBop Showdown is a real-time gamified fashion competition where 4-8 players compete to create the best themed outfit within budget and time constraints. After outfit submission, audiences vote on their favorites, and the highest-voted outfit wins.

Core Game Flow:

1. Host creates a game room with theme, budget, and time limit
2. Players join using a shareable room code
3. Players browse Shopbop catalog and select items
4. Time expires → Voting phase begins automatically
5. Players and audience vote on anonymized outfits
6. Winner announced with full outfit details

2. Technology Stack

We chose a serverless architecture because it fits our capstone timeline, scales automatically, and is cost-effective with AWS education credits.

2.1 Frontend Layer

Component	Technology	Rationale
Framework	React 18+	Modern, component-based
State Management	Context API / Zustand	Simpler than Redux
HTTP Client	Axios	Easy promise-based requests
Real-time	WebSocket (Phase 2)	Live updates

2.2 Backend Layer

Component	Technology	Purpose
API Gateway	AWS API Gateway	REST + WebSocket endpoints

Compute	Lambda (Node.js 18+)	Serverless business logic
Database	DynamoDB	NoSQL serverless storage
Monitoring	CloudWatch	Logs and metrics

3. Database Design

We use Amazon DynamoDB as our database. It fits our serverless AWS setup and requires no server management. There are 4 tables that support game sessions, player management, outfit submissions, and voting.

3.1 Games Table

Tracks the state of each game room — players, status, budget, and timing.

Field	Type	Description
gameId (PK)	String	Unique game identifier (UUID)
hostUserId	String	ID of player who created room
status	String	LOBBY | IN_PROGRESS | VOTING | COMPLETED
theme	String	Style prompt (e.g., Beach Day)
budget	Number	Max spend per outfit ($50-$2000)
maxPlayers	Number	Maximum allowed players (2-8)
currentPlayers	Number	Current player count
timeLimit	Number	Creation time in seconds (60-600)

3.2 Players Table

Field	Type	Description
playerId (PK)	String	Unique player identifier (UUID)
gameId	String	Which game this player belongs to
username	String	Display name shown in game
joinedAt	String	ISO timestamp when player joined
isReady	Boolean	Ready status in lobby

Global Secondary Index: gameId-index (query all players in a game)

3.3 Outfits Table

Field	Type	Description

outfitId (PK)	String	Unique outfit identifier (UUID)
gameId	String	Which game this outfit belongs to
playerId	String	Which player created this outfit
products	List	Array of product objects
totalPrice	Number	Sum of all product prices
votes	Number	Vote count (incremented during voting)

3.4 Votes Table

Field	Type	Description
voteId (PK)	String	Composite: {gameId}#{voterId}
gameId	String	Which game this vote belongs to
voterId	String	Unique identifier for the voter
outfitId	String	Which outfit received this vote
votedAt	String	ISO timestamp when vote was cast

3.5 How Tables Work Together

During a game, players join a session tracked in Games. Each player gets a record in Players linked by gameId. As players build outfits, they're saved to Outfits. When voting begins, voters create records in Votes, and each vote increments the counter in the corresponding Outfits record.

What we need to do	Which table	How
Load a game room	Games	Direct lookup by gameId
Show all players	Players	Query using gameId-index GSI
Show outfits for voting	Outfits	Query by gameId
Record a vote	Votes + Outfits	Insert vote, increment counter
Get game results	Outfits	Query by gameId, sort by votes

4. API Architecture

Our backend exposes REST endpoints through AWS API Gateway, with Lambda functions handling business logic. In Phase 2, we'll add WebSocket support for real-time features.

4.1 Game Management Endpoints

Method	Endpoint	Description
POST	/games	Create new game
GET	/games/{gameId}	Get game details
POST	/games/{gameId}/join	Join existing game
POST	/games/{gameId}/start	Start game (host only)
GET	/games/{gameId}/players	List all players in game

4.2 Outfit Management

Method	Endpoint	Description
POST	/outfits	Submit outfit (with validation)
GET	/games/{gameId}/outfits	Get all outfits for voting

4.3 Voting

Method	Endpoint	Description
POST	/votes	Cast vote (one per voter)
GET	/games/{gameId}/results	Get final ranked results

4.4 Shopbop API Proxy

These endpoints proxy requests to Shopbop API, handling CORS restrictions.

Method	Endpoint	Proxies To
GET	/products/search	Shopbop /public/search
GET	/products/{productSin}	Shopbop /public/products/{id}

GET	/categories	Shopbop /public/folders

5. Implementation Phases

We're building this in 4 phases over 12 weeks to ensure we have a working demo early and can iterate based on mentor feedback.

Phase 1: MVP Foundation (Weeks 1-4)

Goal: Basic working game with REST API only (no real-time features yet).

- Set up AWS infrastructure (DynamoDB, API Gateway, Lambda)
- Core endpoints: Create/join game, submit outfit, vote
- Shopbop API proxy with CORS handling
- Frontend game screens and product browsing
- End-to-end testing of complete game flow

Deliverable: Playable game using polling for state updates.

Phase 2: Real-Time Features (Weeks 5-6)

Goal: Add WebSocket for live game updates.

- WebSocket API Gateway setup and connection management
- Event broadcasting for game state changes
- Real-time timer countdown and live vote updates
- Frontend WebSocket client integration

Deliverable: Real-time game experience without polling.

Phase 3: Polish (Weeks 7-9)

Goal: Production-ready UX and optional advanced features.

- Loading states, error messages, mobile responsive design
- Stretch goals: Add to cart integration, game history
- End-to-end testing and bug fixes
- Performance optimization

Phase 4: Demo Preparation (Weeks 10-12)

- Create demo script and practice presentation
- Prepare backup video in case of technical issues
- Finalize documentation and code comments
- Final mentor review and feedback incorporation

6. Technical Decisions & Rationale

6.1 Database: DynamoDB vs RDS

Criteria	DynamoDB (✓ Chosen)	RDS (Postgres)
Serverless fit	✅Native integration	□ Aurora Serverless only
Auto-scaling	✅Built-in	□ Manual configuration
Cost model	✅Pay per request	□ Always-on pricing
Query flexibility	□ Limited (NoSQL)	✅Full SQL support
Learning curve	□ NoSQL patterns	✅Familiar SQL

Decision: DynamoDB — Mentors strongly recommended it for serverless projects.

6.2 Real-Time: WebSocket vs Polling

Approach	Pros	Cons
Polling (Phase 1)	Simple, familiar	Higher latency, more API calls
WebSocket (Phase 2)	True real-time, better UX	More complex, harder to debug

Decision: Start with polling for MVP, add WebSocket in Phase 2.

6.3 CORS Handling

Approach	Pros	Cons
Lambda per endpoint	Clear separation, easy debug	More functions to manage
API Gateway proxy	Elegant, fewer functions	Complex configuration

Decision: Lambda per endpoint — Easier for team to understand and maintain.

7. Security & Validation

7.1 Input Validation Rules

Field	Validation	Error Message
budget	$50 - $2000	Budget must be between $50 and $2000
timeLimit	60 - 600 seconds	Time must be 1-10 minutes
maxPlayers	2 - 8	Max players must be 2-8
outfit.totalPrice	≤ game.budget	Outfit exceeds budget
vote	One per voter	You have already voted

7.2 API Security

- HTTPS only — All API calls over TLS
- CORS configured for allowed frontend origins
- Rate limiting via API Gateway (1000 req/sec)
- Input sanitization on all endpoints
- DynamoDB encryption at rest (enabled by default)

8. Monitoring & Cost Management

8.1 CloudWatch Monitoring

Metric	Alert Threshold	Action
Lambda errors	> 5% error rate	Investigate logs immediately
Lambda duration	> 5 seconds	Optimize or increase timeout
API Gateway 5xx	> 1%	Check Lambda and DynamoDB health
DynamoDB throttles	> 0	Increase read/write capacity

8.2 Cost Estimation

AWS Free Tier Coverage (12 months):

- Lambda: 1M requests/month, 400K GB-seconds compute
- DynamoDB: 25 GB storage, 25 read/write capacity units
- API Gateway: 1M REST API calls
- S3: 5 GB storage, 20K GET requests

Estimated Monthly Cost: $0-5 (within free tier + education credits)

9. Known Limitations

- No user authentication — Anyone can join with any username
- No persistent user profiles — Players tied to individual games only
- Basic voting — No ranked voting or advanced features
- Polling latency in Phase 1 — 2-3 second delay (resolved in Phase 2)
- Single region deployment — US-East-1 only

10. Resources & Documentation

- JIRA Board: https://amazon-shopbop-showdown.atlassian.net/jira/software/projects/SCRUM/boards/1/backlog
- GitHub Repository: https://github.com/Buman-Erdem/ShopBop-Showdown
- Figma Prototype: https://www.figma.com/make/RWUGQhyVSKv42Y8ibQWZB4/
- AWS Lambda Guide: docs.aws.amazon.com/lambda/
- DynamoDB Guide: docs.aws.amazon.com/dynamodb/
- Shopbop API Base: https://api.shopbop.com
- Client-Id: Shopbop-UW-Team1-2024